Universität Siegen / Naturwissenschaftlich-Technische Fakultät

Eminent
Embedded Intelligence Nanosystems Engineering

Decentralized Edge Intelligence for LoRaWAN: Federated Learning for Environment-Driven Path Loss and Link Quality Modeling

Presented by:

Pratik Khadka

Supervisor:

First Supervisor: Kristof

Second Supervisor: Nahshon M. Obiri

LoRaWAN

Old project:

➢ ***LoRaWAN in Indoor Office Environments: Environmental Effects on Path Loss and Signal Reliability***

- 6 × Arduino **MKR WAN 1310** nodes in an indoor office, with **SCD41, BME280, SPS30** sensors.
- Collected **≈1,328,334 rows** of data over ~4 months: temperature, humidity, CO_2, pressure, PM2.5, plus RSSI, SNR, SF.
- Trained centralized models (MLR, RF, XGBoost) to predict **path loss**. Best: **XGBoost, R^2 ≈ 0.93, RMSE ≈ 5.14 dB**.
- Distilled a **lightweight neural network** using pseudo-labels from XGBoost and converted it to **TensorFlow Lite** for edge deployment (TinyML).
- Built a **Flutter dashboard** with MQTT + PostgreSQL for real-time visualization (but the main intelligence still lived in the server training).

➢ **So: old project = "centralized train-once model + visualization", not a continuously adapting network.**

New thesis: core concept of the upgrade

- **Title direction:**
  "Decentralized Edge Intelligence for LoRaWAN: Federated Learning for Environment-Driven Path Loss and Link Quality Modeling."

Main change:

- **From >centralized data → single model**
- **To >distributed on-device learning → federated aggregation** (TinyML on each node + FL rounds on the backend).

Key Analogy:

- Each node is a **student** with its own notes (local data).
- They send the teacher **what they learned** (model updates, not raw data).
- The teacher combines updates into a better shared guide (global model) and occasionally sends it back.

Feature options for the TinyML model:

- We designed **two feature-set variants** for the node's model:

1. Pure local features (no gateway feedback~downlink)
Only what the node truly knows, free to sample at high rate:

- **Environmental:** temp, humidity, CO_2, pressure, PM2.5.
- **Radio config:** SF, TX power, uplink interval / mode.
- **Local history:**
  - Moving average of recent predictions.
  - Time since last downlink.
  - Count of missed expected downlinks (if you use confirmed uplinks).

This variant works even if you never use downlinks for link metrics.

Feature options for the TinyML model:

2. With occasional feedback (coarse network view)

Because TTN allows **~10 downlinks/day/device**, we treat gateway feedback as **coarse, slow feedback**:

- Aggregated uplink RSSI (e.g., average over last N packets).
- Aggregated uplink SNR.
- Packet delivery ratio (PDR) summary.
- Optional network “link state” flags or ADR-like SF recommendations.

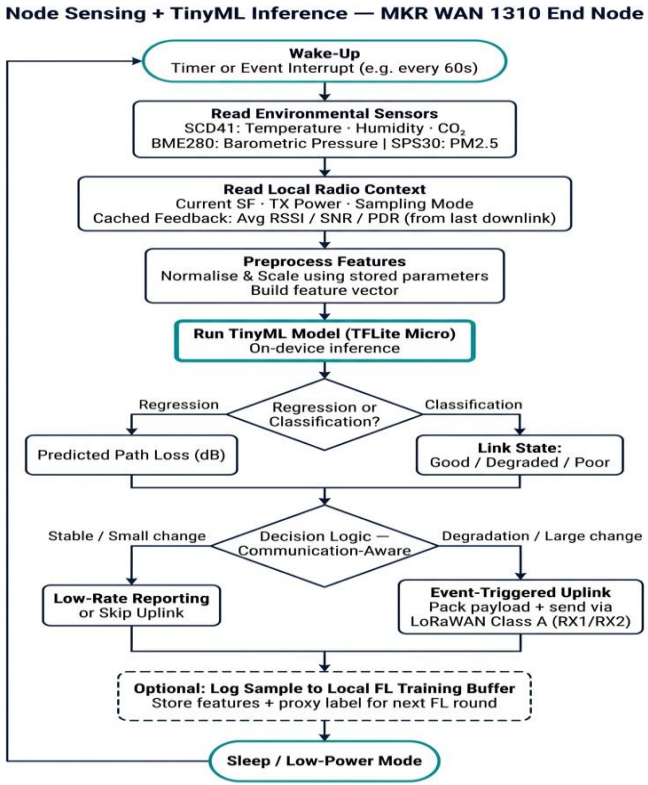
These arrive rarely, so they **cannot be the main training signal**, but they can:

- Calibrate thresholds (“what counts as degraded”),
- Provide a slow “network view” feature for the TinyML model.

We explicitly agreed: 10 such feedback points/day are **too few for dense training**, but **fine for slow calibration and extra context**.

TinyML on the node: what it actually does:

- **Run a small model on-device to predict current path loss / link state and make communication-aware decisions (when/how often to transmit), and to produce local model updates for federated learning.**



Where the ground truth / labels come from now

We clarified a critical point: **nodes do not need the old 1.3M-row dataset during deployment.**

- The old dataset is used **offline** to:
  - Train a strong centralized **teacher** (XGBoost).
  - Distill an initial TinyML **student** model.
  - Provide central **baselines** (accuracy, RMSE) for your evaluation.

During live FL, nodes use **proxy labels**, e.g.:

- For **link state classification**:
  - "Good" = downlink/ACK received.
  - "Degraded" or "poor" = repeated missed downlinks, low PDR.
- For **path loss regression** (harder):
  - Use TX power minus occasional downlink RSSI as a rough path-loss estimate, or
  - Occasionally query server inference (teacher model) and treat those as pseudo-labels.

So live FL is **self-contained**, using online proxy labels; old data is for **pretraining and benchmarking**, not for continuous streaming

Where the ground truth / labels come from now

We clarified a critical point: **nodes do not need the old 1.3M-row dataset during deployment.**

- The old dataset is used **offline** to:
  - Train a strong centralized **teacher** (XGBoost).
  - Distill an initial TinyML **student** model.
  - Provide central **baselines** (accuracy, RMSE) for your evaluation.

During live FL, nodes use **proxy labels**, e.g.:

- For **link state classification**:
  - "Good" = downlink/ACK received.
  - "Degraded" or "poor" = repeated missed downlinks, low PDR.
- For **path loss regression** (harder):
  - Use TX power minus occasional downlink RSSI as a rough path-loss estimate, or
  - Occasionally query server inference (teacher model) and treat those as pseudo-labels.

So live FL is **self-contained**, using online proxy labels; old data is for **pretraining and benchmarking**, not for continuous streaming

Where the ground truth / labels come from now

We clarified a critical point: **nodes do not need the old 1.3M-row dataset during deployment.**

- The old dataset is used **offline** to:
  - Train a strong centralized **teacher** (XGBoost).
  - Distill an initial TinyML **student** model.
  - Provide central **baselines** (accuracy, RMSE) for your evaluation.

During live FL, nodes use **proxy labels**, e.g.:

- For **link state classification**:
  - "Good" = downlink/ACK received.
  - "Degraded" or "poor" = repeated missed downlinks, low PDR.
- For **path loss regression** (harder):
  - Use TX power minus occasional downlink RSSI as a rough path-loss estimate, or
  - Occasionally query server inference (teacher model) and treat those as pseudo-labels.

So live FL is **self-contained**, using online proxy labels; old data is for **pretraining and benchmarking**, not for continuous streaming

Literature anchors:

- **FL + TinyML on IoT MCUs** (surveys, techniques).

**** Y. Hasan *et al.*, "Federated learning and TinyML on IoT edge devices: Challenges, advances, and future directions," *ICT Express*, vol. 11, no. 4, pp. 754–768, Aug. 2025.
https://www.sciencedirect.com/science/article/pii/S2405959525000839
**** A. Casolo and S. Russo, "Federated learning for IoT devices: Enhancing TinyML with on-board training," *Information Fusion*, vol. 104, p. 102189, 2024.
https://dl.acm.org/doi/10.1016/j.inffus.2023.102189
**** M. Alrifai *et al.*, "A Survey on Federated Learning for TinyML: Challenges, Techniques, and Future Directions," *Zenodo*, Apr. 2025, doi: 10.5281/zenodo.15240508.
https://zenodo.org/records/15240508

- **FL over LoRa / LoRaWAN** (case studies, simulators).

**** O. Torres Sanchez *et al.*, "Federated Learning framework for LoRaWAN-enabled IIoT communication: A case study," *arXiv*, Oct. 2024, arXiv:2410.11612.
https://arxiv.org/abs/2410.11612
**** N. Llisterri Giménez *et al.*, "Embedded federated learning over a LoRa mesh network," *Pervasive and Mobile Computing*, vol. 93, p. 101819, Jun. 2023.
https://www.sciencedirect.com/article/pii/S1574119223000779

- **TinyML on MCUs** (on-device inference, distillation, compression).

**** S. Heydari and Q. H. Mahmoud, "Tiny Machine Learning and On-Device Inference: A Survey of Real-World Applications," *Sensors*, vol. 25, no. 10, p. 3191, May 2025, doi: 10.3390/s25103191.
https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
**** N. N. Alajlan and D. M. Ibrahim, "TinyML: Enabling of Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices for AI Applications," *Micromachines*, vol. 13, no. 6, p. 851, May 2022, doi: 10.3390/mi13060851.
https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/

End of Presentation.
Thank you for your attention!

Obiri, N. M., & Van Laerhoven, K. (2024). A Comprehensive Data Description for LoRaWAN Path Loss Measurements in an Indoor Office Setting. IEEE Access. DOI: 10.1109/ACCESS.2024.0429000